Technical Blueprint: Adversarial Jailbreak Detection for Large Language Models

Mrunal Mohan Vibhute
Applied Deep Learning
Project Deliverable 1.2 – Technical Blueprint

---

a. Problem Context and Project Summary

Large language models (LLMs) remain vulnerable to jailbreak attacks: carefully crafted prompts that bypass built-in safety mechanisms and elicit harmful or restricted outputs. This project builds a two-stage detection pipeline that intercepts such prompts before they reach any LLM interface, combining a FAISS semantic cache for sub-millisecond lookup of known attack patterns with an adversarially trained transformer classifier for novel ones. A mutator module generates paraphrased and role-play-wrapped variants of known jailbreaks each training round, continuously hardening the classifier against unseen attack strategies. The system is deployed as a browser extension and a Flask web interface, making it accessible without any changes to the underlying LLM.

b. Dataset

The primary dataset is the **In-The-Wild Jailbreak Prompts** corpus [1], available on HuggingFace under `TrustAIRLab/in-the-wild-jailbreak-prompts`. It contains real jailbreak prompts collected from community sources (Reddit, Discord, dedicated jailbreak repositories) through December 2023, with binary labels (1 = jailbreak, 0 = benign). The dataset loads directly via the HuggingFace `datasets` library with no manual download required.

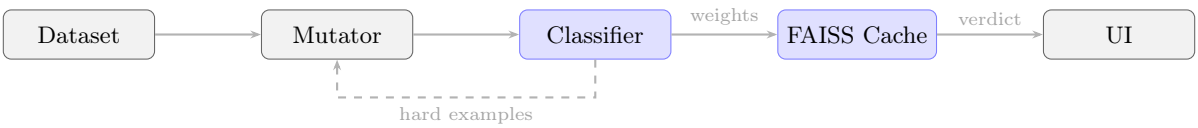
Format and access: Plain text prompts with integer labels. Loaded as a HuggingFace `DatasetDict` and converted to Pandas for EDA. Split 80/10/10 (train/validation/test) using stratified sampling to preserve class ratios.

Preprocessing challenges: The dataset exhibits class imbalance, with jailbreak prompts forming a minority of the total. This is handled via class-weighted cross-entropy loss during training. Deduplication is applied before splitting to prevent data leakage. The mutator augments the training set with synthetic variants each adversarial round.

Out-of-distribution test set: Incorporating feedback from Deliverable 1.1, a small held-out set of 50–100 post-December 2023 jailbreak examples will be sourced from more recent community repositories (e.g., JailbreakBench, WildTeaming) to evaluate genuine generalization beyond the training distribution.

Ethical considerations: The dataset consists of publicly collected jailbreak prompts used solely for defensive detection research. No personally identifiable information is present. The system is designed to protect users, not to facilitate attacks. All experiments are conducted on a local/Colab environment with no live LLM API calls.

c. Planned Architecture



Three transformer architectures will be benchmarked for the classifier: DistilBERT-base (fast baseline), ModernBERT-base (2024, longer context window), and DeBERTa-v3-small (strong classification benchmark). All are fine-tuned via HuggingFace `transformers` with AdamW and a linear warmup schedule. A TF-IDF +

Logistic Regression baseline is included for comparison. The FAISS semantic cache uses `all-MiniLM-L6-v2` embeddings with cosine similarity over L2-normalised vectors.

d. User Interface Plan

The interface consists of two components. The **Flask web app** (`ui/app.py`) provides a text area where a user types or pastes a prompt, clicks "Check Prompt," and receives an immediate colour-coded verdict (red for jailbreak, green for benign) along with the detection stage (cache or model), confidence score, and latency. The **browser extension** runs the same detection call as a content script, intercepting form submissions on LLM websites before the prompt is sent.

Inputs: Free-text prompt entered by the user.

Outputs: Binary verdict (jailbreak / benign), detection stage, confidence percentage, and response latency in milliseconds.

Wireframe sketch: The interface shows a single text area at the top, a "Check" button below it, and a result panel beneath that. The result panel changes background colour based on verdict and displays the metadata fields in plain text. No login, no history, no external calls – deliberately minimal.

Interpretability: The confidence score and stage field make it transparent whether the decision came from the fast cache or the neural model, helping users understand why a prompt was flagged.

e. Innovation and Anticipated Challenges

The key distinguishing contribution of this project is the **adversarial co-evolution loop**: rather than training a static classifier on a fixed dataset, the mutator and classifier co-evolve across rounds, with the classifier continuously hardened against mutations it generates itself. The two-stage architecture adds an operationally meaningful evaluation dimension (cache hit rate vs. model call rate) beyond standard F1. The runtime context injection module allows new attack vocabulary to be added post-training without fine-tuning.

Challenge 1: Cache update feedback loop. Incorporating feedback from Deliverable 1.1, automatic cache updates from classifier detections risk compounding false positives over time. Mitigation: a confidence threshold of 0.85 is required for automatic cache updates; lower-confidence detections go into a manual review queue rather than being cached immediately.

Challenge 2: Mutation quality gate calibration. The cosine similarity threshold at 0.6 that discards low-quality mutations requires empirical justification. Mitigation: before running the full adversarial loop, a manual audit will be conducted across thresholds of 0.5, 0.6, and 0.7 on a sample of 50 mutations to verify that the chosen value preserves jailbreak intent while varying surface form.

Challenge 3: Out-of-distribution generalization. A classifier hardened against mutations of December 2023 attacks may fail on structurally novel strategies. Mitigation: a held-out OOD test set of 50–100 post-2023 examples will be sourced from JailbreakBench and WildTeaming to stress-test generalization explicitly.

f. Implementation Timeline

Week	Focus	Expected Outcome
Apr 1–5	Environment setup, dataset loading, EDA	Working `setup.ipynb`, EDA plots, repo committed
Apr 5–12	Baseline classifier (DistilBERT), TF-IDF baseline, data preprocessing pipeline	First F1 score on test set, clean data loader
Apr 12–19	Mutator integration, adversarial training loop (2–3 rounds), mutation quality audit	Measurable F1 improvement after adversarial rounds
Apr 19–26	FAISS cache integration, two-stage detector, Flask UI, OOD evaluation	End-to-end pipeline running, UI serving predictions
Apr 26–29	ModernBERT and DeBERTa benchmarking, final evaluation, report writing	Comparison table, final PDF report, presentation ready

g. Responsible AI Reflection

Fairness: The dataset was collected from English-language communities, meaning the classifier may underperform on jailbreaks written in other languages or culturally specific phrasing. This limitation will be noted explicitly in the final report.

Transparency: The detection system surfaces its confidence score and decision stage to the user, making it auditable rather than a black box. The manual review queue for low-confidence cache updates adds a human-in-the-loop safeguard.

Misuse risk: The mutator module generates jailbreak variants for the purpose of adversarial training. To prevent misuse, it is not exposed through any public endpoint and is used solely within the controlled training loop.

Environmental cost: Fine-tuning transformer models on Colab T4 GPUs is low-cost relative to large-scale training runs. Each adversarial round takes approximately 20–30 minutes; the full experiment is estimated at under 5 GPU hours total.

References

[1] Shen, X. et al. (2023). *"In-the-Wild" Jailbreak Prompts on Large Language Models*. HuggingFace Datasets. https://huggingface.co/datasets/TrustAIRLab/in-the-wild-jailbreak-prompts

[2] Hackett, W. et al. (2025). *Machine Learning for Detection and Analysis of Novel LLM Jailbreaks*. arXiv:2510.01644.

[3] Mo, Y. et al. (2024). *Fight Back Against Jailbreaking via Prompt Adversarial Tuning*. arXiv:2402.06255.

[4] Chao, P. et al. (2024). *JailbreakBench: An Open Robustness Benchmark for Jailbreaking LLMs*. NeurIPS 2024 Datasets and Benchmarks Track.